

Solving Recurrence Relations: Recursion Tree and Master Theorem

1. Introduction

When analyzing the time complexity of divide-and-conquer algorithms (like Merge Sort or Quick Sort), we often encounter recurrence relations of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where:

- $a \geq 1$ is the number of subproblems,
- n/b is the size of each subproblem,
- $f(n)$ is the cost of dividing and combining the results.

We will learn two methods to solve such recurrences:

1. Recursion Tree Method
2. Master Theorem

2. Recursion Tree Method

This method involves drawing the recursion as a tree and summing the cost at each level.

Example:

Solve the recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Step 1: Draw the tree.

Each call splits into two subproblems of size $n/2$, each costing $n/2$ for the combine step.

- Level 0: n
- Level 1: $2 \cdot \frac{n}{2} = n$

- Level 2: $4 \cdot \frac{n}{4} = n$
- ...
- Level $\log_2 n$: $2^{\log_2 n} \cdot \frac{n}{2^{\log_2 n}} = n$

Each level costs n , and there are $\log_2 n + 1$ levels.

$$T(n) = n \cdot (\log_2 n + 1) = \Theta(n \log n)$$

Conclusion:

The total work at each level is n . Since there are $\log n$ levels, the overall time is $\Theta(n \log n)$.

3. Master Theorem

The Master Theorem gives an easier way to solve recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Theorem:

Let $a \geq 1$, $b > 1$, and $f(n)$ be an asymptotically positive function. Then:

1. If $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$, then:

$$T(n) = \Theta(n^{\log_b a})$$

2. If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$, then:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$, and if the regularity condition holds:

$$af\left(\frac{n}{b}\right) \leq cf(n) \text{ for some } c < 1 \text{ and sufficiently large } n,$$

then:

$$T(n) = \Theta(f(n))$$

4. Examples

Example 1:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Here, $a = 2$, $b = 2$, $f(n) = n$

$$n^{\log_2 2} = n \Rightarrow f(n) = \Theta(n)$$

So, this is case 2 $\Rightarrow T(n) = \Theta(n \log n)$

Example 2:

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

Here, $a = 4$, $b = 2$, $f(n) = n^2$, and:

$$n^{\log_2 4} = n^2 \Rightarrow f(n) = \Theta(n^2)$$

Case 2 applies $\Rightarrow T(n) = \Theta(n^2 \log n)$

Example 3:

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Here, $a = 4$, $b = 2$, so:

$$n^{\log_2 4} = n^2, \quad f(n) = n = O(n^{2-1})$$

Case 1 applies $\Rightarrow T(n) = \Theta(n^2)$

5. Summary

- Use the recursion tree to understand how work is distributed across recursive calls.
- Use the Master Theorem for quick solutions when applicable.
- Know the form: $T(n) = aT(n/b) + f(n)$
- Always verify whether Master Theorem conditions are satisfied.

Further Reading

- CLRS (Introduction to Algorithms), Chapter 4
- GeeksForGeeks: Master Method
- CMU Lecture Slides on Recurrences